DataSim: A Friendly Simulation Lab for Relational Database Systems

Akshat Raj Vishal Vivek Mohit Kumar

26 September 2026

Course: Database Management Systems
Repository: https://github.com/albertaj23/DataSim

Contributors

Reg. No.	Name
24BCE0043	Akshat Raj
24BCE2377	Vishal Vivek
24BCE2378	Mohit Kumar

Contents

1 Project Title **3**

2 Introduction and Background **3**

3 Problem Statement **3**

4 Objectives **4**

5 Scope of the Project **4**

5.1 Included 4

5.2 Excluded 4

6 Requirement Analysis **5**

6.1 Users 5

6.2 Data to be stored 5

6.3 Required operations 5

6.4 Information to retrieve and analyze 6
6.5 Functional requirements 6
6.6 Non-functional requirements 6

7 Database Selection and Justification **6**
7.1 Selected technology 6

8 Data Model / Database Design **7**
8.1 Main entities and attributes 7
8.2 Entity-relationship diagram 7
8.3 Relational schema 7
8.4 Keys and constraints 8
8.5 Normalization 8

9 System Architecture / Workflow Diagram **9**
9.1 High-level architecture 9
9.2 Scenario workflow 9
9.3 System output 9

1 Project Title

DataSim: A General-Purpose Simulation Lab for High-Impact Database Scenarios.

Scenario family: Shared-resource workflows covering inventory, reservations, payments, banking, collaboration, job processing, and quota management.

2 Introduction and Background

Database systems support applications where many users and clients read and modify shared data at the same time. A database may appear correct during a single-user demonstration but behave incorrectly when concurrent requests arrive together. Examples include lost updates, inconsistent reads, race conditions, lock waits, deadlocks, and constraint violations.

DataSim is a general-purpose interactive database simulation lab that makes these behaviors visible. Like a focused learning simulator such as Cisco Packet Tracer, it provides approachable interfaces for constructing and observing scenarios without requiring users to manage every low-level detail manually. DataSim supports repeatable scenarios and technical views for examining transactions, locks, constraints, isolation levels, and results across domains that depend on shared state.

The project targets high-impact database situations: preventing inventory overselling, allocating the last seats or rooms fairly, making payment and order requests idempotent, maintaining consistency during bank transfers, merging collaborative edits, assigning jobs without duplication, and enforcing quotas or rate limits. These scenarios share the same underlying problems of concurrent reads, writes, ownership, allocation, and recovery. Future scenarios can reuse the platform's controls, database instrumentation, experiment runner, and result visualizations.

3 Problem Statement

The initial implementation demonstrates a shared-resource race in which several clients belonging to one owner may attempt to claim a limited resource at nearly the same time. The same pattern appears when customers buy the last product, travelers reserve the last seat, workers claim the same job, or two services update one account simultaneously. Each request can read the same database state before writing. If the operations are not coordinated correctly, the system may oversell inventory, double-allocate a resource, lose an update, charge twice, or violate a quota. DataSim keeps the database techniques domain-independent so the same controls can represent any of these situations.

The project addresses the following problem:

> How can a relational DBMS maintain a correct, real-time shared state under concurrent operations, and what correctness and performance trade-offs are introduced by different transaction, isolation, locking, and constraint strategies?

DataSim also demonstrates the lost-update problem. The same anomaly can affect stock counts, account balances, document versions, counters, and job status when concurrent read-modify-write operations overwrite one another.

4 Objectives

1. Model shared users, resources, operations, events, and experiment results in a normalized relational database.
2. Demonstrate concurrency anomalies using reproducible scenarios rather than only theoretical examples.
3. Compare naive operations, transactions, serializable isolation, pessimistic locking, optimistic version checks, unique constraints, triggers, and a Redis lease strategy.
4. Show how isolation levels affect visibility, blocking, deadlocks, and correctness.
5. Visualize transaction schedules, lock information, wait-for relationships, rollbacks, and invariant checks.
6. Simulate scalable groups of virtual users using real database transactions across reusable scenario patterns.
7. Provide a friendly interface for learners and a technical interface for detailed inspection.
8. Persist experiment configurations and results so that runs can be compared and analyzed.
9. Keep the platform extensible so future scenarios can reuse the same simulation and measurement patterns.

5 Scope of the Project

5.1 Included

- MySQL 8.4 relational database using the InnoDB storage engine.
- A relational schema for users, resources, allocations, operations, events, and experiment runs.
- Shared-resource limits, leases, and expiry rules that can represent quotas, reservations, inventory, or work allocation.
- Multiple concurrency-control strategies for the same business operation.
- Lost-update experiments using a shared counter and other read-modify-write patterns.
- Transaction Stepper for statement-by-statement execution of two transactions.
- Concurrency Lab for repeated race experiments and measurement.
- Simulation Control Room for virtual users and real database traffic.
- InnoDB lock and wait-for information where available.
- Invariant checks for quotas, leases, duplicate allocations, and audit consistency.
- React web interface, Node.js/TypeScript API, Docker setup, and automated tests.

5.2 Excluded

- Real domain services such as inventory fulfillment, payment processing, banking, reservations, collaboration, or content storage; DataSim models database behavior rather than operating these services.

- Production authentication, authorization, subscriptions, or payment processing.
- Production-scale cloud deployment or guaranteed multi-region availability.
- Personalization, recommendations, advertising, and social networking.
- Any claim that the implementation represents the internal design of a commercial banking, retail, reservation, collaboration, or payment service.
- Unlimited arbitrary workloads; the simulator uses bounded, controlled scenarios for repeatability.

6 Requirement Analysis

6.1 Users

User	Main needs
Student or learner	Understand DBMS concepts through guided visual scenarios.
Demonstrator or instructor	Run repeatable experiments and explain anomalies.
Technical user	Inspect SQL effects, locks, transactions, constraints, and results.
Developer	Add new database strategies or scenarios without replacing the platform.

6.2 Data to be stored

- **User/account:** identity, quota or limit, conflict policy, state version, and laboratory flag.
- **Client:** owner, name/type, and last-seen time.
- **Resource:** descriptive attributes, availability, capacity, and shared counters.
- **Allocation/operation:** user, resource, status, quantity, timestamps, lease, and strategy.
- **Operation event:** state transition, references, timestamp, version, request id, and details.
- **Experiment run:** type, strategy, configuration, outcomes, latency, throughput, violations, and batch id.

6.3 Required operations

1. Look up a user/account and its clients or owned resources.
2. Create, update, reserve, release, or cancel a shared resource operation.
3. Renew a lease or heartbeat for an in-progress operation.
4. Resolve conflicts according to a selected policy.
5. Reject expired, preempted, duplicate, or stale client requests.
6. Switch the active concurrency strategy.
7. Run quota, allocation, and lost-update experiments.
8. Step through two transactions one statement at a time.

9. Start, pause, resume, repair, and stop a crowd simulation.
10. Inspect live state, audit events, locks, and experiment history.

6.4 Information to retrieve and analyze

The system retrieves active sessions per account, invariant status, lost updates, transaction outcomes, retries, deadlocks, rollbacks, lock waits, latency percentiles, throughput, violation counts, event timelines, and strategy comparisons.

6.5 Functional requirements

1. The system shall store user, client, resource, allocation, event, and experiment data.
2. The system shall support concurrent claims, updates, allocations, or reservations from multiple users or clients.
3. The system shall implement and compare multiple protection strategies.
4. The system shall record experiment configurations and results.
5. The system shall expose invariant and consistency checks.
6. The system shall provide friendly scenario controls and technical inspection views.
7. The system shall reject expired, preempted, duplicate, or stale operations.

6.6 Non-functional requirements

The system must provide correctness, repeatability, observability, bounded scalability, maintainability, local portability, and usability. Experiments must be controlled and their results must be persisted for later comparison.

7 Database Selection and Justification

7.1 Selected technology

- **Database:** MySQL 8.4.
- **Type:** Relational database management system.
- **Storage engine:** InnoDB.
- **Additional component:** Redis 7 for an optional lease-based comparison strategy.
- **Application stack:** Node.js, TypeScript, Express, React, and Socket.IO.

A relational DBMS is appropriate because **DataSim** studies transactions, isolation levels, row and gap locks, constraints, foreign keys, deadlocks, and consistency of related entities. MySQL/InnoDB provides these mechanisms in a real database engine, allowing the project to measure actual behavior rather than simulate every rule in application code.

MySQL was selected because InnoDB supports ACID transactions and multiple isolation levels; row-level and next-key locking; foreign keys, unique constraints, checks, triggers, and generated columns; performance-schema lock inspection; and reproducible Docker-based execution. Redis is included as a separate comparison point to show how an external

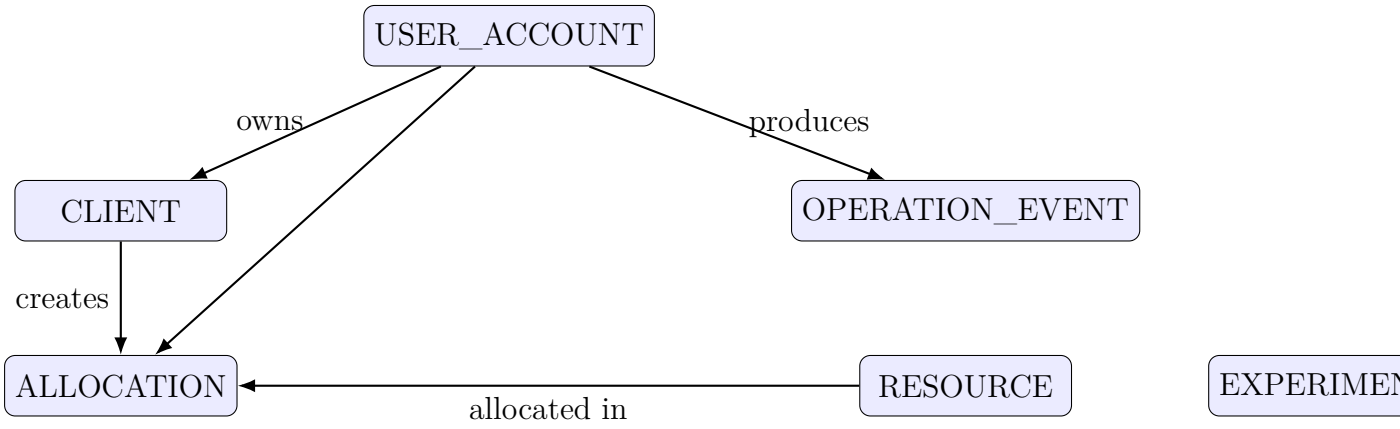
key-value lease gate can coordinate access while MySQL remains the durable relational source of truth.

8 Data Model / Database Design

8.1 Main entities and attributes

Entity	Important attributes	Keys/constraints
`user_account`	id, username, quota, conflict policy, state version, lab flag	PK, unique username, checks
`client`	id, account, name, type, last seen	PK, FK, unique account/name
`resource`	id, name, type, capacity, shared counter	PK, non-negative values
`allocation`	client, account, resource, status, quantity, lease, strategy	PK, composite FK, resource FK, indexes
`operation_event`	state transition, references, version, request id, details	PK, unique request id
`experiment_run`	type, strategy, config, outcomes, latency, throughput	PK, batch indexes

8.2 Entity-relationship diagram



8.3 Relational schema

```
USER_ACCOUNT(account_id PK, username UNIQUE, quota, conflict_policy,
       state_version, is_lab)

CLIENT(client_id PK, account_id FK -> USER_ACCOUNT, client_name,
       client_type, last_seen_at, UNIQUE(account_id, client_name))
```

```
RESOURCE(resource_id PK, resource_name, resource_type, capacity,
         shared_counter)

ALLOCATION(allocation_id PK, client_id, account_id, resource_id FK -> RESOURCE,

           status, quantity, started_at, lease_expires_at,
           ended_at, strategy,
           FK(client_id, account_id) -> CLIENT(client_id, account_id))

OPERATION_EVENT(event_id PK, account_id, client_id, allocation_id, event_type,
                state_version, client_request_id, created_at, detail,
                UNIQUE(client_id, client_request_id))

EXPERIMENT_RUN(run_id PK, experiment, strategy, config, outcomes,
              violations, p50_ms, p95_ms, throughput, batch_id, created_at)
```

8.4 Keys and constraints

- Primary keys identify every user, client, resource, allocation, event, and run.
- Foreign keys preserve ownership and reference integrity.
- A composite foreign key ensures that an allocation's client belongs to its recorded account.
- Unique username and account/client-name constraints prevent duplicate identities.
- A unique client/request-id constraint makes state-changing requests idempotent.
- Status, policy, quota, quantity, capacity, and counter checks reject invalid values.
- An optional generated-column unique index demonstrates declarative protection for one active allocation per account.
- Allocation limits are predicates over multiple rows, so they require concurrency control rather than only a primary key.

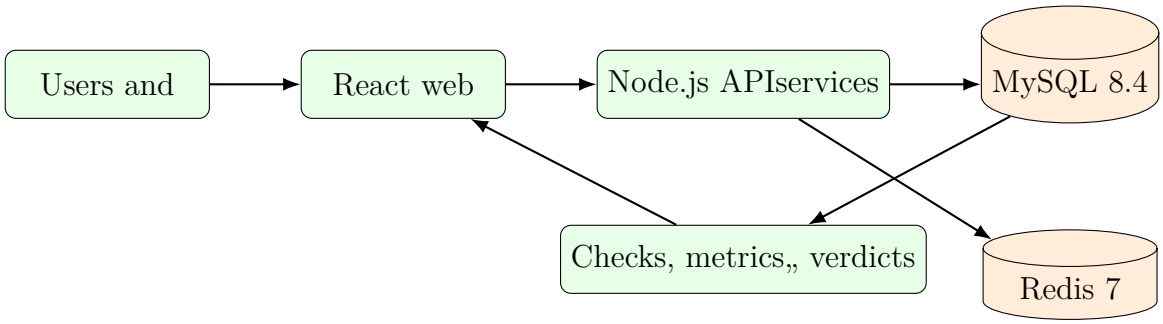
Core invariant pattern: For every shared owner or resource, active allocations and consumed quantity must remain within the configured quota or capacity.

8.5 Normalization

The core schema follows third normal form. Each table represents one main entity or event type; attributes are atomic; non-key attributes depend on the whole key; and client, resource, event, and experiment details are not unnecessarily duplicated. The allocation table intentionally stores both `client_id` and `account_id` because owner-level queries and locking are central to the workload. The composite foreign key keeps this controlled denormalization consistent.

9 System Architecture / Workflow Diagram

9.1 High-level architecture



9.2 Scenario workflow

1. A user selects a scenario and concurrency strategy in the React interface.
2. The interface sends an operation, experiment, or simulation request to the Node.js API.
3. The service begins a MySQL transaction and reads the relevant shared state.
4. The selected strategy applies locking, isolation, optimistic checks, constraints, or an external lease.
5. MySQL returns a grant, rejection, wait, deadlock, rollback, or conflict result.
6. The service commits or rolls back, records an event or experiment result, and publishes committed state.
7. The interface displays updated sessions, metrics, checks, timelines, and technical diagnostics.

9.3 System output

The system produces friendly scenario and simulation views, live user/client/resource state, strategy verdicts, transaction schedules, lock and wait information, experiment history, latency and throughput measurements, violation counts, and CSV/Markdown results for analysis.

Conclusion

DataSim is designed as a general-purpose simulation lab for studying database behavior under concurrency. It addresses high-impact patterns such as overselling, double allocation, duplicate payment requests, inconsistent transfers, lost edits, duplicate job claims, and quota violations. Its domain-neutral entities and diagrams make the project applicable to many systems without exposing one particular business domain at the high level. By connecting friendly interfaces to real MySQL transactions and measurable outcomes, the project shows how database design choices affect correctness, performance, and scalability across different systems.